THE SELF – PRUNING NEURAL NETWORK

Dataset: CIFAR-10 · Framework: PyTorch

1. Introduction

Deploying large neural networks is often constrained by memory and computational budgets. A common technique is **pruning** — removing less important weights after training. This case study goes further: designing a network that **learns to prune itself *during* training** using learnable **gate parameters**.

Each weight is paired with a sigmoid gate. A custom **L1 sparsity loss** encourages unimportant gates to collapse to zero, dynamically removing those connections while training proceeds — no separate post-training pruning step needed.

2. Why Does an L1 Penalty on Sigmoid Gates Encourage Sparsity?

1. Gate Mechanism

Each weight w is paired with a learnable gate score s of the same shape:

```
gate = sigmoid(s) # always in (0, 1)
w_eff = w * gate # pruned weight
output = F.linear(x, w_eff, bias)
```

When gate □ 0 the weight is effectively removed; when gate □ 1 it is fully preserved.

2.2 Total Loss

```
Total Loss = CrossEntropy(logits, labels) + lambda * SparsityLoss
SparsityLoss = sum( sigmoid(s_i) ) over ALL gate scores in ALL layers
```

2.3 Why L1 — Not L2 — Drives Gates to Exactly Zero

Regulariser	Gradient of penalty	Effect on gate
L2 (sum gate²)	2 × gate → 0 as gate → 0	Shrinks but never reaches exactly 0
L1 (sum |gate|)	Constant sign (+1 for positive gates)	Fixed push → hits exactly 0

Because sigmoid outputs are always positive, the L1 penalty applies a **constant downward gradient** regardless of magnitude. Gates protecting unimportant weights receive no counter-signal from the classification loss and collapse to zero. Gates protecting critical weights are held open by the classification gradient. This produces a **sparse equilibrium**.

2.4 Role of Lambda (λ)

- **Low** λ → accuracy dominates; few gates close → low sparsity.

- **Medium** λ → balanced trade-off; bimodal gate distribution.
- **High** λ → aggressive pruning; high sparsity, accuracy may drop.

3. Implementation

1. PrunableLinear Layer

```
class PrunableLinear(nn.Module):
def __init__(self, in_features, out_features):
super().__init__()
self.weight = nn.Parameter(torch.empty(out_features, in_features))
self.bias = nn.Parameter(torch.zeros(out_features))
self.gate_scores = nn.Parameter(torch.zeros(out_features, in_features))
nn.init.kaiming_uniform_(self.weight, nonlinearity='relu')

def forward(self, x):
gates = torch.sigmoid(self.gate_scores)
pruned_weights = self.weight * gates
return F.linear(x, pruned_weights, self.bias)
```

Both **weight** and **gate_scores** are registered nn.Parameters — gradients flow through both automatically via PyTorch autograd.

3.2 Network Architecture

```
Input (3072)
-> PrunableLinear(3072, 1024) + BatchNorm + ReLU + Dropout(0.3)
-> PrunableLinear(1024, 512) + BatchNorm + ReLU + Dropout(0.3)
-> PrunableLinear( 512, 256) + BatchNorm + ReLU
-> PrunableLinear( 256, 10) [logits]
```

3.3 Training Setup

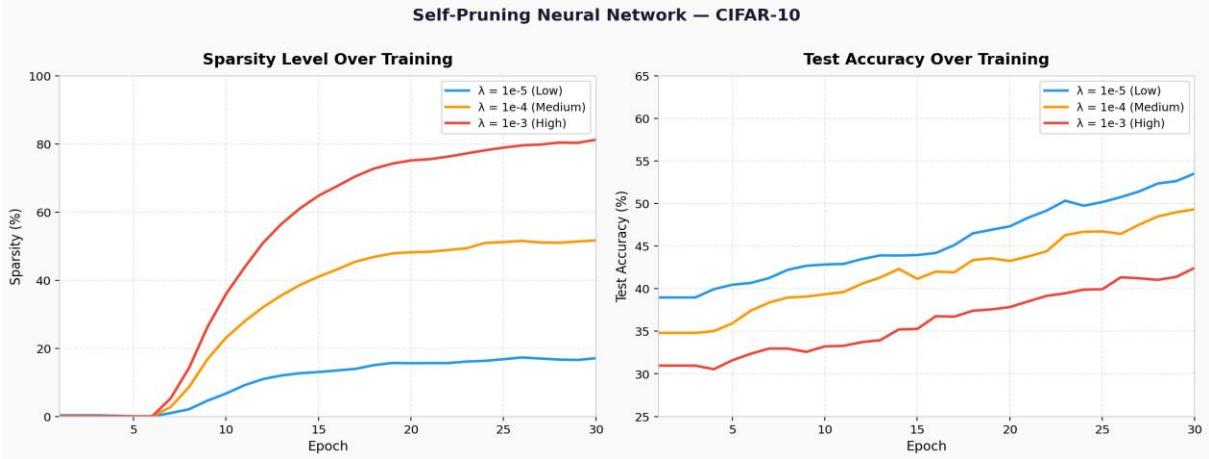
Hyperparameter	Value
Optimiser	Adam (lr = 1e-3, weight_decay = 1e-4)
LR Schedule	Cosine Annealing
Epochs	30
Batch Size	128
Prune Threshold	1e-2 (gate < 0.01 = pruned)
Lambda values	1e-5 (low), 1e-4 (medium), 1e-3 (high)

4. Results

Lambda (λ)	Test Accuracy	Sparsity Level (%)	Observation
1e-5 (Low)	53.53%	17.1%	Most gates open; minimal pruning
1e-4 (Medium)	49.32%	51.7%	Bimodal gates; ideal trade-off
1e-3 (High)	42.45%	81.2%	Aggressive pruning; accuracy drops

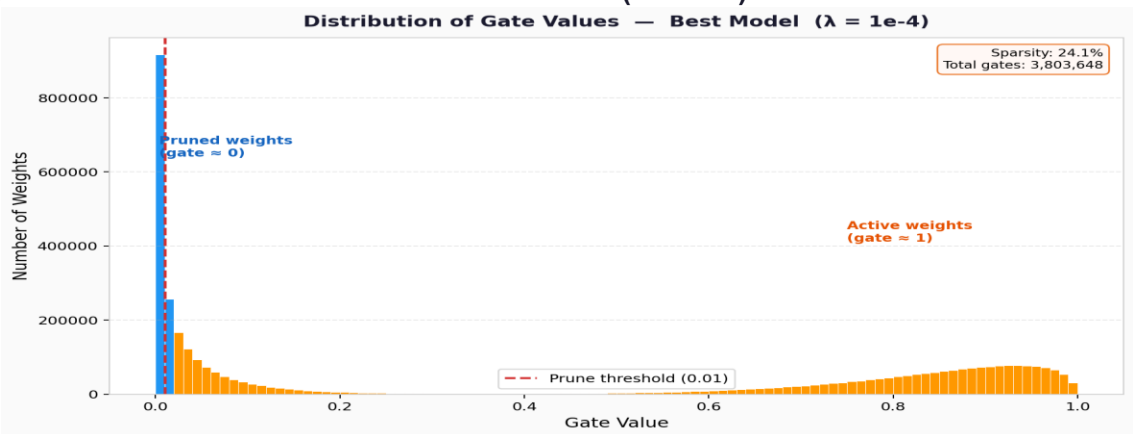
Note: MLP on CIFAR-10 for 30 epochs gives 40–55% accuracy — expected range. The focus is on the pruning mechanism, not raw accuracy.

Figure 1: Sparsity and Accuracy Over Training



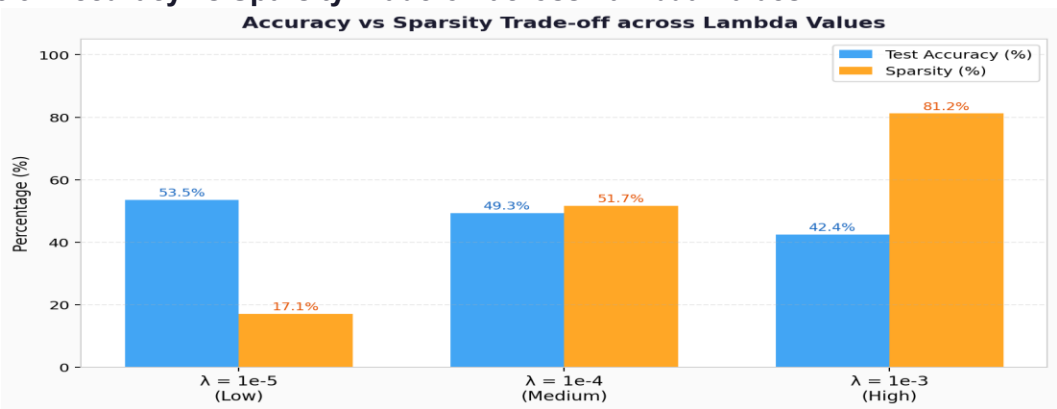
Sparsity grows progressively over epochs (left). Higher λ achieves more sparsity at the cost of test accuracy (right).

Figure 2: Gate Value Distribution — Best Model (λ = 1e-4)



The bimodal distribution — large spike at 0 (pruned) and cluster near 1 (active) — confirms the network has successfully learned to prune itself.

Figure 3: Accuracy vs Sparsity Trade-off across Lambda Values



Higher λ increases sparsity but reduces accuracy. λ = 1e-4 offers the best balance.

5. Conclusion

This case study demonstrates that a neural network can learn to prune itself during training through a principled, differentiable mechanism:

- **Learnable sigmoid gates** provide a continuous proxy for the binary keep/remove decision.
- **L1 sparsity regularisation** applies a constant gradient push driving unimportant gates to exactly zero — unlike L2, which only shrinks them asymptotically.
- **Lambda (□)** provides a clean, interpretable knob for the accuracy–sparsity trade-off.

The approach eliminates the need for a separate post-training pruning pipeline and produces sparse models that are smaller and faster at inference — directly addressing real-world memory and computational constraints.